

[✦ Upgrade](#)

I need to document my Vibe Coding process for a Lab-4 submission.

Project: High-Low Card Predictor using Pygame.

I modified game/game_engine.py to:

1. Fix the card comparison bug by changing rank_str comparisons to numeric_rank comparisons.
2. Add self.streak = 0.
3. Increase the streak after a correct prediction.

Show more ▾



Absolutely — here's a **Lab-4 submission-ready Vibe Coding process** based only on the changes you actually made to `game/game_engine.py`. No unrelated features or invented work.

Lab-4 — Vibe Coding Process Documentation

Lab-4: Vibe Coding Process Documentation

Project: High-Low Card Predictor using Pygame

1. Project Overview

The project is a **High-Low Card Predictor game developed using Pygame**. The player predicts whether the next card will be higher or lower than the current card using the **HIGHER** and **LOWER** buttons.

For Lab-4, the existing game was modified using a Vibe Coding approach. The work was restricted to:

`game/game_engine.py`

The main objectives were to fix an existing card comparison bug and introduce a streak-based scoring mechanism while preserving the existing game functionality.

2. Understanding the Existing Code

The first step was to understand how the existing game engine worked before making any modifications.

The `game/game_engine.py` file was inspected to identify:

- How cards were represented.
- How card ranks were compared.
- How predictions were evaluated.
- How the existing score was updated.
- How incorrect predictions affected the score.
- How the HIGHER and LOWER buttons were connected to the game logic.
- Where the score was displayed.

The existing implementation compared card ranks using their string representation. This created a problem because card ranks represented as strings do not always produce the correct numerical ordering.

For example, comparing values as strings can result in incorrect ordering because string comparison is based on character ordering rather than the actual numerical value of the card.

3. Identifying the Existing Bug

The main bug identified in the existing implementation was the use of `rank_str` for comparing card ranks.

The comparison logic needed to determine the actual numerical relationship between the current card and the next card.

Therefore, the existing comparison was changed from using the string representation of the rank to using the corresponding `numeric_rank` value.

Before

The comparison logic relied on:

```
rank_str
```

After

The comparison uses:

```
numeric_rank
```

This ensures that card values are compared according to their actual numerical order.

4. Planning the Changes

Before implementing the changes, the required modifications were broken down into smaller steps.

The planned changes were:

1. Keep the existing card prediction functionality.
2. Replace string-based rank comparison with numerical rank comparison.
3. Add a streak counter to the game engine.
4. Increase the streak after a correct prediction.
5. Implement streak-based scoring:
 - Streak below 3 → 1 point
 - Streak 3–4 → 2 points
 - Streak 5 or higher → 3 points
6. Reset the streak after an incorrect prediction.
7. Preserve the existing score reduction for incorrect predictions.
8. Display the current streak below the score.
9. Preserve the existing HIGHER and LOWER buttons.
10. Modify only `game/game_engine.py`.

This kept the implementation focused on the requirements without changing unrelated parts of the project.

5. Implementing the Changes

5.1 Adding the Streak Variable

A new streak variable was added immediately after the existing score initialization.

```
self.score = 0
self.streak = 0
```

This initializes the player's streak to zero when the game starts.

5.2 Fixing Card Comparison

The card comparison logic was modified to use the numerical rank of the cards instead of comparing their string representations.

The logic now evaluates the actual numerical values represented by `numeric_rank`.

This prevents incorrect comparisons caused by treating card ranks as strings.

5.3 Updating the Streak After a Correct Prediction

When the player's prediction is correct, the streak is incremented:

```
self.streak += 1
```

The streak therefore represents the number of consecutive correct predictions.

5.4 Implementing Streak-Based Scoring

The number of points awarded for a correct prediction depends on the current streak.

The scoring rules implemented were:

Streak	Points Awarded
Below 3	1
3–4	2
5 or higher	3

This creates progressively higher rewards for maintaining a longer sequence of correct predictions.

5.5 Handling an Incorrect Prediction

When the prediction is incorrect, the existing score reduction behavior is preserved.

At the same time, the streak is reset:

```
self.streak = 0
```

This ensures that consecutive correct predictions are required to maintain the streak-based bonus.

The incorrect-prediction behavior was therefore extended rather than replacing the existing scoring behavior.

6. Updating the Game Display

The game interface was updated to display the current streak below the score.

The display now communicates both:

Score: X

Streak: X

This allows the player to see their current consecutive-correct-prediction count while playing.

The existing **HIGHER** and **LOWER** buttons were preserved and were not removed or replaced.

7. Debugging

After implementing the changes, the modified `game/game_engine.py` was reviewed for possible syntax and logic issues.

The debugging process focused on:

- Checking the new `streak` variable initialization.
- Checking that the streak increments only after a correct prediction.
- Checking that the correct scoring range is selected based on the streak.
- Checking that an incorrect prediction resets the streak.
- Ensuring the existing score reduction remains intact.
- Ensuring the numerical card comparison uses `numeric_rank`.
- Ensuring the **HIGHER** and **LOWER** controls remain unchanged.
- Ensuring no unrelated files were modified.

The implementation was then tested after the changes were completed.

8. Testing

Two verification commands were used to validate the implementation.

8.1 Python Syntax Verification

The modified Python file was compiled using:

```
python -m py_compile game/game_engine.py
```

The command completed successfully.

This verified that `game/game_engine.py` contained valid Python syntax and did not contain syntax errors.

8.2 Git Diff Verification

The repository was also checked using:

```
git diff --check
```

This completed successfully.

This verified that there were no whitespace errors or formatting problems detected by Git in the changes.

9. Git Verification

After testing, the Git changes were reviewed to ensure that the intended project modification was captured correctly.

The modification was limited to:

```
game/game_engine.py
```

The purpose of the Git verification was to confirm that the changes corresponded to the planned card-comparison fix and streak-based scoring functionality rather than introducing unrelated modifications.

10. Committing the Changes

Once the implementation and verification were completed, the changes were committed to Git.

The commit message used was:

```
Add streak-based scoring
```

This commit records the implementation of the streak-based scoring changes along with the related card comparison fix.

11. Pushing to GitHub

After the commit was created successfully, the changes were pushed to the personal GitHub repository.

This ensured that the completed Lab-4 implementation was available in the remote repository and that the local Git history contained the completed change.

12. Final Implementation Summary

The final implementation in `game/game_engine.py` contains the following changes:

- Fixed the card comparison bug by using `numeric_rank` instead of `rank_str`.
- Added a `streak` variable initialized to `0`.
- Increased the streak after a correct prediction.
- Added streak-based scoring:
 - Below 3 → 1 point
 - 3–4 → 2 points
 - 5 or higher → 3 points
- Reset the streak to `0` after an incorrect prediction.
- Preserved the existing score reduction after an incorrect prediction.
- Added a visible **Streak: X** display below the score.
- Preserved the existing **HIGHER** and **LOWER** buttons.
- Modified only `game/game_engine.py`.
- Verified the Python file using `py_compile`.
- Verified Git changes using `git diff --check`.
- Committed the changes as:

```
Add streak-based scoring
```

- Pushed the completed changes to the personal GitHub repository.

13. Vibe Coding Workflow

The overall Vibe Coding process followed this sequence:

Understand Existing Code

↓

Identify Card Comparison Bug

↓

Define Required Streak Features

↓

Plan Minimal Changes

↓

Modify `game/game_engine.py`

↓

Debug and Review Logic

↓

Run Python Syntax Check

↓

Run Git Diff Check

↓

Commit Changes

↓

Push to GitHub

The implementation followed a **minimal-change approach**, focusing only on the required behavior in `game/game_engine.py` while preserving the existing game controls and functionality.

This is written so you can use it directly as the **Lab-4 Vibe Coding process section**. If your submission needs screenshots, the natural places to insert them are the **implementation, testing, Git verification, commit, and GitHub push** sections.



Original Bug: `rank_str` vs `numeric_rank`

The original bug was in the **card comparison logic**. The game was comparing the card ranks using `rank_str`, which stores the rank as a **string** rather than as a numerical value.

For example, if card ranks are represented as:

```
"2", "3", "4", ..., "9", "10", "J", "Q", "K", "A"
```

string comparison does **not** represent the actual numerical order of cards. Strings are compared character-by-character, so values such as `"10"` can behave differently from the numerical value `10`.

The game needs to answer questions like:

Is the next card's rank higher or lower than the current card's rank?

For that, the ranks need to be compared according to their **actual card values**, not their text representation.

Why `numeric_rank` was needed

`numeric_rank` provides a numerical representation of each card's rank, allowing comparisons such as:

```
2 < 10 < 13 < 14
```

instead of relying on string ordering.

So the comparison was changed from conceptually:

Python

Run

```
current_card.rank_str < next_card.rank_str
```

to:

Python

Run

```
current_card.numeric_rank < next_card.numeric_rank
```

This makes the HIGHER/LOWER prediction logic use the actual numerical ordering of the cards.

In short: `rank_str` is useful for **displaying** the card rank, while `numeric_rank` is appropriate for **calculating and comparing** card values.



Explain the streak scoring implementation and the UI change.

Streak Scoring Implementation

A **streak system** was added to reward the player for making consecutive correct predictions.

First, a new variable was initialized:

Python

```
self.streak = 0
```

Run

This means the player starts with a streak of zero.

When the player makes a **correct prediction**, the streak is increased:

Python

```
self.streak += 1
```

Run

The number of points awarded is then determined by the current streak:

Streak	Points
1–2	+1
3–4	+2
5+	+3

So, for example:

- First correct prediction → streak = 1 → **+1 point**
- Second correct prediction → streak = 2 → **+1 point**
- Third correct prediction → streak = 3 → **+2 points**
- Fifth correct prediction → streak = 5 → **+3 points**

When the player makes an **incorrect prediction**, the streak is reset:

Python

Run

```
self.streak = 0
```

The existing score reduction for an incorrect prediction is still retained. This means the streak is specifically tracking **consecutive correct predictions**, while the original penalty system continues to work.

UI Change

The game's interface was also updated to make the streak visible to the player.

Previously, the score was displayed, but there was no indication of the player's current consecutive-correct count.

A new display was added **below the score**:

```
Score: X  
Streak: X
```

For example:

```
Score: 7  
Streak: 4
```

This gives immediate feedback about the player's current streak and lets them understand when they are approaching the higher scoring levels.

The existing **HIGHER** and **LOWER** buttons were preserved, so the UI change only adds the streak information without changing the existing prediction controls.



Explain the debugging, testing, Git verification, commit, and GitHub push.

Debugging

After implementing the card comparison fix and streak system, the modified `game/game_engine.py` was reviewed to make sure the new logic worked together with the existing game logic.

The debugging focused on checking that:

- `self.streak` was initialized correctly to `0`.
- The streak increased only after a **correct prediction**.
- The correct number of points was awarded based on the streak.
- The streak reset to `0` after an **incorrect prediction**.
- The existing score reduction for an incorrect prediction was preserved.
- `numeric_rank` was being used for card comparisons.
- The existing **HIGHER** and **LOWER** buttons were not affected.
- No unnecessary project files were modified.

The goal was to fix the required functionality without changing unrelated parts of the game.

Testing

After debugging, the modified Python file was checked for syntax errors using:

Bash

```
python -m py_compile game/game_engine.py
```

The command completed successfully.

This confirmed that `game/game_engine.py` had valid Python syntax and could be compiled by Python.

Git Verification

The changes were then checked using:

Bash

```
git diff --check
```

This also completed successfully.

This check verified that the Git changes did not contain whitespace or formatting errors.

The changes were also verified to ensure that the modification was limited to the intended file:

```
game/game_engine.py
```

Commit

Once the implementation and checks were successful, the changes were committed to Git.

The commit message was:

```
Add streak-based scoring
```

This created a permanent entry in the project's Git history describing the implemented change.

GitHub Push

Finally, the committed changes were pushed to the user's personal GitHub repository.

The push transferred the local commit to the remote repository, making the completed Lab-4 implementation available on GitHub.

So the complete verification workflow was:

```
Modify code
  ↓
Debug and review
  ↓
python -m py_compile game/game_engine.py
  ↓
git diff --check
  ↓
Git commit
"Add streak-based scoring"
  ↓
Push to personal GitHub repository
```

This demonstrates the complete Vibe Coding workflow from **implementation** → **debugging** → **testing** → **Git verification** → **version control** → **remote GitHub submission**.



ChatGPT can make mistakes. Check important info.